

KUKA youBot

Методичка по запуску симуляции и задачи
детекции, захвата и перестановки кубика по цвету

ROS Noetic · Gazebo · RViz · Ubuntu 20.04

Часть I — запуск симуляции с генератором комнаты (`room_generator`)

Часть II — детекция цвета, захват и перестановка (`youbot_perception`)

Оглавление

Документ состоит из двух частей. Часть I поднимает симуляцию, комнату и точки A/B/C. Часть II добавляет камеру, детекцию цвета кубика, захват и постановку на картонку соответствующего цвета. Часть II опирается на окружение, подготовленное в Части I.

ЧАСТЬ I. Запуск симуляции с визуализацией (room_generator)

- 1 Подготовка: библиотеки и инструменты
- 2 Сборка проекта и права на узлы
- 3 Запуск с визуализацией — порядок панелей
- 4 Параметры комнаты и точки B
- 5 Настройка RViz
- 6 Частые проблемы (Часть I)

ЧАСТЬ II. Детекция, захват и перестановка кубика по цвету (youbot_perception)

- 7 Что делает решение
- 8 Дополнительная подготовка: зрение и Gazebo-grasp
- 9 Установка пакета и сборка
- 10 Быстрый запуск (один launch)
- 11 Проверка по ходу запуска
- 12 Параметры запуска
- 13 Запуск на реальном роботе
- 14 Топики решения (шпаргалка)
- 15 Частые проблемы (Часть II)

ЧАСТЬ I

Запуск симуляции с визуализацией

ROS Noetic · Gazebo · RViz · Ubuntu 20.04. Инструкция «с нуля»: предполагается, что ROS и Gazebo уже установлены, но проект ещё ни разу не запускался. Версия с генератором комнаты (нода `room_generator`): препятствия, сетка и точки A/B/C строятся автоматически и воспроизводимо.

1. Подготовка: библиотеки и инструменты

Выполняется один раз. Требуется интернет (Ethernet-кабель или USB-модем).

1.1. Терминал Terminator

Удобен тем, что одно окно делится на несколько панелей (нам понадобится 5).

```
sudo apt update
sudo apt install terminator -y
```

Разбиение на панели: **Ctrl+Shift+E** — вертикально, **Ctrl+Shift+O** — горизонтально. Переход между панелями: **Ctrl+Shift+стрелки**.

1.2. ROS-контроллеры (управление колёсами, рукой, схватом)

Без них контроллеры не загрузятся и робот не поедет (ошибка «controller type does not exist»).

```
sudo apt install ros-noetic-ros-control ros-noetic-ros-controllers \
ros-noetic-gazebo-ros-control ros-noetic-velocity-controllers \
ros-noetic-effort-controllers ros-noetic-joint-state-controller \
ros-noetic-position-controllers ros-noetic-joint-trajectory-controller -y
```

1.3. Навигация и телеуправление

```
sudo apt install ros-noetic-navigation ros-noetic-teleop-twist-keyboard \
ros-noetic-gmapping ros-noetic-amcl ros-noetic-map-server -y
```

1.4. Python-библиотеки (нужны планировщику траектории)

Планировщик использует `scipy` и `numpy` для сглаживания путей сплайнами. Без них узел `youbot_controller` падает с ошибкой «No module named 'scipy'».

```
sudo apt install python3-scipy python3-numpy -y
```

Важно. Никогда не устанавливайте пакет `catkin` командой `sudo apt install catkin` — он ломает зависимости ROS. Команда `catkin_make` уже входит в ROS Noetic; если её «не видно», просто подключите окружение: `source /opt/ros/noetic/setup.bash`.

2. Сборка проекта (один раз)

Если проект ещё не собран — соберите. Если уже собирали, переходите к разделу 3.

```
cd ~/youbot_ws
source /opt/ros/noetic/setup.bash
catkin make
source ~/youbot_ws/devel/setup.bash
```

2.1. Права на исполняемые узлы (обязательно!)

Исполняемыми должны быть ЧЕТЫРЕ Python-узла (добавился `room_generator.py`). После замены файлов флаг прав слетает, поэтому выставляем его вручную:

```
chmod +x ~/youbot_ws/src/RMPC-project-KUKA-youBot/src/youbot_mecanum_controller/scripts/mecanum_controller.py
chmod +x ~/youbot_ws/src/RMPC-project-KUKA-youBot/src/youbot_controller/scripts/youbot_controller.py
chmod +x ~/youbot_ws/src/RMPC-project-KUKA-youBot/src/youbot_controller/scripts/visualization_node.py
```

```
chmod +x ~/youbot_ws/src/RMPC-project-KUKA-youbot/src/youbot_controller/scripts/room_generator.py
```

Только эти четыре `.py` файла. Не выполняйте `chmod` на папки целиком или на файлы `.bash` / `.yaml` / `.launch` — это путает запуск. Файл `setup.bash` всегда подключается через `source`, а не запускается напрямую.

3. Запуск с визуализацией — порядок панелей

Открываем Terminator, делим на 5 панелей. В каждой панели первыми двумя строками подключаем окружение (`source ...`). Запускаем строго по порядку: 1 → 2 → ждём Gazebo → 3 → 4 → 5.

Панель 1 — ROS master (roscore)

Запускаем первой. Ждём строку «started core service». Больше не трогаем.

```
source /opt/ros/noetic/setup.bash
roscore
```

Панель 2 — Симуляция + контроллер + генератор комнаты (Gazebo)

Этот launch теперь поднимает и `room_generator` (строит комнату, сетку, точки A/B/C и виртуальные препятствия). Ждём, пока откроется окно Gazebo и в логе появятся строки генератора комнаты, а затем — лог контроллера.

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch youbot_controller youbot_sim.launch
```

В логе сначала должны появиться строки `room_generator`, например:

```
Комната 5.00 x 5.00 м, сетка 5 x 5 (ячейка 1.00 x 1.00 м), seed=42
Точки: A(park)=(0.50, 0.50) B(pick)=(2.50, 2.50) C(place)=(4.50, 4.50)
Стенок-цилиндров: ... виртуальных препятствий: ...
```

Затем побегут строки контроллера с фазами миссии и целями B → C → A (точка кубика, склад, возврат на парковку):

```
t=..., phase=APPROACH, goal=(2.50,2.50), ...
```

О точке В (левый верхний угол). По умолчанию В — в центре комнаты. Чтобы поставить кубик в левый верхний угол, передайте аргумент при запуске (см. раздел 4.1). Координаты в логе изменятся соответственно.

Панель 3 — Проверка одометрии (контрольная точка)

Запускаем, когда Gazebo поднялся. Должно показать частоту ~50 Hz.

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
rostopic hz /odom
```

Идёт ~50 Hz → Ctrl+C и дальше. «no new messages» → СТОП: проблема в симуляции (проверьте, что узлы из п. 2.1 исполняемые), дальше нет смысла.

Панель 4 — Узел визуализации (маркеры миссии)

Публикует путь робота (`/robot_path`) и подписи точек A/B/C. Должно появиться «Mission visualization node started» и «Старт зафиксирован».

Изменение. Сами препятствия, стенки, сетка и точки A/B/C теперь публикует `room_generator` в топик `/room/markers` (он уже запущен из Панели 2). Этот узел больше не дублирует препятствия — поэтому в RViz добавляем оба топика маркеров (см. раздел 5).

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch youbot_controller visualization_node.py
```

Панель 5 — RViz (3D-визуализация)

Если Gazebo и RViz конкурируют за видеокарту и экран зависает — запустите RViz на программном рендеринге (вторая строка-альтернатива).

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch rviz rviz
# если всё виснет — вместо строки выше:
# LIBGL_ALWAYS_SOFTWARE=1 roslaunch rviz rviz
```

4. Параметры комнаты и точки В

Размеры комнаты, сетка, зерно ГСЧ и расположение точек A/B/C задаются аргументами launch. Размер комнаты на реальном роботе будет другим — достаточно поменять аргументы, код менять не нужно. Результат воспроизводим за счёт seed.

4.1. Точка В — два способа

Способ 1 — по умолчанию (центр комнаты). Ничего не указываем, В встаёт в центральную ячейку:

```
roslaunch youbot_controller youbot_sim.launch
```

Способ 2 — левый верхний угол. Передаём ячейку В в формате `col,row`. Нумерация: столбцы слева направо начиная с 0, ряды снизу вверх начиная с 0 (начало координат в левом-нижнем углу). Значит верхний ряд — это `n_rows - 1`. Для комнаты 5×5 (сетка 5×5) левый верхний угол — это `0,4`:

```
roslaunch youbot_controller youbot_sim.launch cell_B:=0,4
```

Подсказка. Если поменяете размер комнаты, индекс верхнего ряда тоже изменится. Например, для комнаты с сеткой 4 ряда левый верхний угол — это `0,3`. Узнать число рядов: `rosparam get /room/n_rows`.

4.2. Другие полезные аргументы

Аргумент	По умолч.	Назначение
room_width	5.0	Ширина комнаты n (м), вдоль X
room_height	5.0	Высота комнаты m (м), вдоль Y
seed	42	Зерно ГСЧ — воспроизводимость препятствий
cell_B	(центр)	Ячейка точки В (кубик), формат <code>col,row</code>

Пример комбинации (комната 3×4, В в левом верхнем углу, другое зерно):

```
roslaunch youbot_controller youbot_sim.launch \
room_width:=3.0 room_height:=4.0 cell_B:=0,3 seed:=7
```

5. Настройка RViz (один раз)

После того как окно RViz открылось:

- 1. Global Options → Fixed Frame: выбрать **odom**
- 2. Add → By display type → **RobotModel** — модель робота
- 3. Add → By topic → `/odom` → Odometry — стрелка позиции
- 4. Add → By topic → `/room/markers` → MarkerArray — комната, стенки, сетка, препятствия, точки A/B/C
- 5. Add → By topic → `/mission_markers` → MarkerArray — подписи миссии
- 6. Add → By topic → `/robot_path` → Path — линия пути робота
- 7. Add → By topic → `/scan_front` → LaserScan — лучи лидара

Новое. Главный топик с препятствиями теперь `/room/markers`. Если в RViz не видно комнаты и препятствий — почти всегда забыт именно этот пункт 4.

Сохранить настройку: File → Save Config As → `~/youbot_ws/mission.rviz`. В следующий раз: `roslaunch rviz rviz -d ~/youbot_ws/mission.rviz`.

Если в RViz пусто: почти всегда дело в Fixed Frame — он должен быть **odom**, а не map. Фрейм `odom` появляется в списке только когда идёт одометрия (проверка в Панели 3).

6. Частые проблемы (Часть I)

Симптом	Решение
<code>roscore / catkin_make</code> not found	Не подключено окружение: <code>source /opt/ros/noetic/setup.bash</code> . Не ставить предложенные системой пакеты.
Permission denied на <code>setup.bash</code>	Файл запущен напрямую. Подключать только через <code>source</code> (с пробелом).
<code>cannot launch node ... chmod +x</code>	Узел не исполняемый. Выполнить <code>chmod +x</code> из п. 2.1. Не забудьте <code>room_generator.py</code> .
В RViz нет комнаты / препятствий	Не добавлен топик <code>/room/markers</code> (раздел 5, п. 4) или Fixed Frame не <code>odom</code> .
Робот едет не в ту точку В	Проверьте аргумент <code>cell_B</code> и лог <code>room_generator</code> («B(pick)=...»).
<code>controller type ... does not exist</code>	Не установлены ros-контроллеры — см. п. 1.2. Перезапустить симуляцию.
<code>No module named 'scipy'</code>	Установить: <code>sudo apt install python3-scipy python3-numpy -y</code> (п. 1.4).
RViz / экран виснет	Gazebo и RViz конкурируют за GPU: <code>LIBGL_ALWAYS_SOFTWARE=1 roslaunch rviz rviz</code> .
<code>No p gain ... caster_joint_*</code>	Не критично (касторные ролики). Игнорировать.

Порядок запуска Части I. Панель 1 (roscore) → Панель 2 (Gazebo + room_generator) → ждать загрузки → Панель 3 (проверка `/odom`) → Панель 4 (маркеры пути) → Панель 5 (RViz, Fixed Frame = odom, топик `/room/markers`).

ЧАСТЬ II

Детекция, захват и перестановка кубика по цвету

Запуск пакета `youbot_perception`. Робот приезжает в точку В, по камере определяет цвет кубика (red/green/blue), захватывает его, везёт в точку С и ставит на картонку соответствующего цвета. Тот же код работает на реальном роботе. Опирается на окружение из Части I.

7. Что делает решение

1. **Детекция.** Узел `cube_detector.py` по RGB-D камере на башне находит кубик, определяет его цвет по HSV-маскам и вычисляет 3D-позицию в кадре `odom`. Публикует `/cube/color` и `/cube/pose`.
2. **Захват и перенос.** Узел `pick_place_mission.py` (конечный автомат) едет в В, ждёт устойчивый цвет, берёт кубик и везёт в С.
3. **Постановка по цвету.** В С лежат три картонки red/green/blue. Робот подъезжает к картонке нужного цвета и ставит кубик (red→red, green→green, blue→blue).
4. **Gazebo + реальный робот.** Узел `spawn_task_objects.py` расставляет в симуляции кубик случайного цвета и картонки; истинный цвет в детектор не передаётся.

8. Дополнительная подготовка: зрение и Gazebo-grasp

Пункты 1.1-1.4 из Части I должны быть уже выполнены. Ниже — то, что нужно добавить специально для задачи с камерой.

8.1. Компьютерное зрение

Для детекции цвета нужен OpenCV + `cv_bridge`, для трансформации точек — `tf2`, для расстановки объектов — `gazebo_msgs`.

```
sudo apt install ros-noetic-cv-bridge ros-noetic-vision-opencv \
ros-noetic-image-transport ros-noetic-tf2-geometry-msgs \
ros-noetic-gazebo-msgs python3-opencv -y
```

8.2. Плагин надёжного захвата

Чтобы кубик надёжно держался в схвате Gazebo (без проскальзывания), нужен плагин `gazebo_grasp_fix`:

```
sudo apt install ros-noetic-gazebo-grasp-plugin -y
# если пакета нет в репозитории — собрать из исходников gazebo-pkgs:
# cd ~/youbot_ws/src && git clone https://github.com/JenniferBuehler/gazebo-pkgs.git
# cd ~/youbot_ws && catkin_make
```

Если плагин поставить не удаётся. Удалите блок `<plugin name="gazebo_grasp_fix">` из файла `youbot_perception/urdf/youbot_task.urdf.xacro`. Тогда захват держится только на трении — менее надёжно, но запуск не сломается.

9. Установка пакета и сборка

9.1. Положить пакет в рабочее пространство

Распакуйте `youbot_perception` в каталог `src`, рядом с остальными пакетами проекта:

```
~/youbot_ws/src/
├─ RMPc-project-KUKA-youBot/      # основной проект (Часть I)
└─ youbot_perception/            # ← новый пакет (Часть II)
```

9.2. Сборка

```
cd ~/youbot_ws
source /opt/ros/noetic/setup.bash
catkin_make
```

```
source ~/youbot_ws/devel/setup.bash
```

9.3. Права на исполняемые узлы (обязательно!)

Кроме четырёх узлов из п. 2.1, исполняемыми должны быть **три новых** Python-узла:

```
chmod +x ~/youbot_ws/src/youbot_perception/scripts/cube_detector.py
chmod +x ~/youbot_ws/src/youbot_perception/scripts/pick_place_mission.py
chmod +x ~/youbot_ws/src/youbot_perception/scripts/spawn_task_objects.py
```

10. Быстрый запуск (один launch)

Весь сценарий — Gazebo, робот с камерой, контроллеры, комната, объекты, детекция и миссия — поднимается одной командой.

Панель 1 — ROS master (опционально)

```
source /opt/ros/noetic/setup.bash
roscore
```

Панель 2 — Полный запуск задачи

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch youbot_perception pick_place_sim.launch
```

Что произойдёт по шагам (видно в логе):

- откроется Gazebo, заспавнится youBot с башенной 3D-камерой;
- поднимутся контроллеры колёс/руки/схвата и `mecanum_controller` (одометрия);
- `room_generator` построит комнату и точки A/B/C;
- через ~6 с `spawn_task_objects` поставит кубик **случайного** цвета в В и три картонки в С;
- `cube_detector` начнёт публиковать цвет и позицию кубика;
- через ~10 с `pick_place_mission` начнёт миссию: В → детекция → захват → С → постановка на картонку нужного цвета → возврат в А.

Признак успеха в логе миссии: строки вида Цвет кубика определён: green, затем Захват кубика (green)..., Целевая картонка green → (...) и в конце Миссия завершена: кубик green установлен на картонку green.

11. Проверка по ходу запуска (контрольные панели)

Эти панели не обязательны, но помогают понять, что всё работает.

Панель 3 — Одометрия

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
rostopic hz /odom
```

Должно быть ~50 Hz. «no new messages» → СТОП: проблема в симуляции (проверьте права из п. 9.3).

Панель 4 — Что видит камера (детекция)

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
# определённый цвет:
rostopic echo /cube/color
# позиция кубика в odom:
rostopic echo /cube/pose
# кадр с разметкой найденного кубика:
rqt_image_view /cube/debug_image
```

В `rqt_image_view` на кубике должен появиться кружок и подпись цвета. Если изображение чёрное/пустое — проверьте: `rostopic hz /tower_cam3d/rgb/image_raw`.

Панель 5 — RViz (по желанию)

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash
roslaunch rviz rviz
# если экран виснет:
# LIBGL_ALWAYS_SOFTWARE=1 roslaunch rviz rviz
```

Fixed Frame = `odom`. Полезные топики: `/room/markers`, `/tower_cam3d/depth_registered/points` (PointCloud2), `/cube/pose` (PoseStamped), RobotModel.

12. Параметры запуска

Размеры комнаты, сетка и точки A/B/C наследуются от `room_generator` (см. раздел 4). Новый launch добавляет свои аргументы.

Аргумент	По умолч.	Назначение
<code>cube_color</code>	(пусто = случайно)	Зафиксировать цвет кубика для отладки: <code>red</code> / <code>green</code> / <code>blue</code>

seed	42	Зерно ГСЧ для воспроизводимости
gui	true	Показывать окно Gazebo (false — без GUI)

Примеры:

```
# Зафиксировать красный кубик (удобно для проверки детекции):
roslaunch youbot_perception pick_place_sim.launch cube_color:=red

# Поставить кубик в левый верхний угол комнаты 5x5 (ячейка B = 0,4):
roslaunch youbot_perception pick_place_sim.launch cell_B:=0,4

# Другая комната и зерно:
roslaunch youbot_perception pick_place_sim.launch \
  room_width:=3.0 room_height:=4.0 cell_B:=0,3 seed:=7
```

Где задаются картонки. По умолчанию три картонки кладутся у точки C со смещением по оси Y: red на **-0.5 м**, green по центру, blue на **+0.5 м**. И `spawn_task_objects.py`, и `pick_place_mission.py` используют одни и те же смещения, поэтому робот едет к правильной картонке. Переопределить можно параметрами `pad_red` / `pad_green` / `pad_blue`.

13. Запуск на реальном роботе

Тот же код, но без Gazebo и без спавна объектов (они реальные). База/рука/схват — через `youbot_driver_ros_interface`; камера — реальный сенсор (Astra/Xtion/RealSense), его драйвер запускается отдельно.

```
source /opt/ros/noetic/setup.bash
source ~/youbot_ws/devel/setup.bash

# 1) отдельно поднять драйвер камеры, публикующий RGB и
#    ВЫРОВНЕННОЕ (registered) облако точек, например:
roslaunch astra_camera astra.launch      # пример

# 2) запустить задачу с указанием реальных топигов камеры:
roslaunch youbot_perception pick_place_real.launch \
  rgb_topic:=/camera/rgb/image_raw \
  points_topic:=/camera/depth_registered/points
```

Перед реальным пуском. Убедитесь, что есть TF от кадра камеры до `odom`; подгоните HSV-пороги в `cube_detector.py` под освещение помещения; уточните позы руки `ARM_*` в `pick_place_mission.py` под фактическую высоту кубика и картонки.

14. Топики решения (шпаргалка)

Топик	Тип	Назначение
/tower_cam3d/rgb/image_raw	sensor_msgs/Image	Вход: RGB-кадр камеры
/tower_cam3d/depth_registered/points	sensor_msgs/PointCloud2	Вход: облако точек (для 3D-позиции)
/cube/color	std_msgs/String	Выход детектора: red / green / blue
/cube/pose	geometry_msgs/PoseStamped	Выход детектора: позиция кубика в odom
/cube/debug_image	sensor_msgs/Image	Кадр с разметкой (для проверки)
/cmd_vel	geometry_msgs/Twist	Команда движения базы
/arm_controller/command	trajectory_msgs/JointTrajectory	Команды руки (5 суставов)
/gripper_controller/command	trajectory_msgs/JointTrajectory	Команды схвата

15. Частые проблемы (Часть II)

Симптом	Решение
cannot launch node ... chmod +x	Узел не исполняемый. Выполнить <code>chmod +x</code> из п. 9.3 (все три новых .py).
No module named 'cv2' / cv_bridge	Не установлены пакеты зрения — см. п. 8.1.
/cube/color молчит, debug-кадр пустой	Камера не публикует: <code>rostopic hz /tower_cam3d/rgb/image_raw</code> . Если тишина — робот заспавлен без камеры (запускайте именно <code>pick_place_sim.launch</code> , он грузит <code>youbot_task.urdf.xacro</code>).
Цвет определяется неверно	Подгоните HSV-диапазоны <code>COLOR_RANGES</code> в начале <code>cube_detector.py</code> под освещение/материалы.
/cube/pose не публикуется, цвет есть	Облако точек не organized или нет TF до <code>odom</code> . Проверьте <code>rostopic hz .../depth_registered/points</code> и <code>roslaunch rqt_tf_tree rqt_tf_tree</code> .
Кубик выскальзывает из схвата	Не установлен <code>gazebo_grasp_fix</code> (п. 8.2). Поставьте плагин или уточните позы <code>ARM_GRASP</code> / <code>GRIPPER_CLOSED</code> .
Робот промахивается мимо картонки	Проверьте, что смещения картонок в <code>spawn_task_objects.py</code> и <code>pick_place_mission.py</code> совпадают. Допуск подъезда — параметр <code>pos_tol</code> .

<code>controller type ... does not exist</code>	Не установлены ros-контроллеры — см. п. 1.2, перезапустить симуляцию.
RViz / экран виснет	<code>LIBGL_ALWAYS_SOFTWARE=1 rosruncvizrviz</code> .

Порядок запуска Части II (итог). Панель 1 (`roscore` , по желанию) → Панель 2 (`roslaunch youbot_perception pick_place_sim.launch`) → дождаться Gazebo и спавна объектов → Панель 3 (`rostopic hz /odom`) → Панель 4 (`rostopic echo /cube/color` , `rqt_image_view /cube/debug_image`) → Панель 5 (RViz, при желании). Миссия стартует автоматически.